

目录

1	工业轴承设备剩余寿命预测系统的实现设计文档	1
1.1	文档信息	1
1.2	版本记录	1
1.3	1. 系统设计目标	1
1.4	2. 总体架构设计	2
1.5	3. 功能模块设计	2
1.5.1	3.1 M1 数据接入与管理模块	2
1.5.2	3.2 M2 预处理与特征工程模块	2
1.5.3	3.3 M3 退化阶段划分与标签构造模块	2
1.5.4	3.4 M4 模型训练与预测模块	2
1.5.5	3.5 M5 评估、可视化与实验管理模块	3
1.6	4. 数据流设计	3
1.7	5. 数据库与数据文件设计	3
1.8	6. 核心算法设计	3
1.8.1	6.1 预处理算法	3
1.8.2	6.2 退化阶段划分算法	3
1.8.3	6.3 预测算法	4
1.9	7. 接口设计	4
1.9.1	7.1 数据接口	4
1.9.2	7.2 训练接口	4
1.9.3	7.3 评估与展示接口	4
1.10	8. 系统运行流程设计	4
1.11	9. 异常处理设计	4
1.12	10. 部署设计	4
1.13	11. 设计总结	5

1 工业轴承设备剩余寿命预测系统的实现设计文档

1.1 文档信息

项目	内容
文档名称	设计文档
文档阶段	mid-term
项目名称	工业轴承设备剩余寿命预测系统的实现
版本	V2.0
编写日期	2026-03-12
文档负责人	zyj
参与编写	cyj、zdh、zy
设计基线	五大模块划分、统一数据抽象、统一训练框架

1.2 版本记录

版本	日期	编写人	说明
V1.0	2025-11-10	zyj、cyj	完成总体架构设计
V2.0	2026-03-12	zyj、cyj、zdh、zy	补充算法、接口、异常和部署设计

1.3 1. 系统设计目标

设计目标不是实现单一模型，而是构建一套结构清晰、职责明确、便于扩展的工业轴承设备剩余寿命预测框架。具体目标如下：

1. 屏蔽不同官方数据集的目录差异，提供统一数据抽象。
2. 将预处理、标签构造、训练、预测、评估和展示解耦。
3. 允许新模型或新算法在不破坏主流程的情况下接入。
4. 支持实验记录、训练监控和结果导出。

1.4 2. 总体架构设计

系统采用自底向上的分层架构：

```
数据接入层
└─ XJTULoader / PHM2012Loader
数据抽象层
└─ BearingEntity / BearingWindowDataset
预处理与特征工程层
└─ SignalProcessor / FeatureExtractor / Pipeline
退化分析与标签构造层
└─ RUL Labeler / Stage Labeler / HI Labeler
模型训练与预测层
└─ Models / Trainer / Tester / Predictor
评估与展示层
└─ Metrics / Evaluator / Visualizer
实验管理层
└─ Tracker / Callback / Serializer
```

该架构的优点在于数据、算法和展示可以独立演化，同时保证主流程仍然稳定。

1.5 3. 功能模块设计

1.5.1 3.1 M1 数据接入与管理模块

该模块负责将不同数据集目录组织映射为统一实体对象。设计要点如下：

1. 通过 BaseBearingLoader 定义统一接口。
2. 具体 loader 负责识别目录、解析文件和补充元数据。
3. BearingEntity 作为统一的数据实体，保存样本、采样率、通道和元信息。

1.5.2 3.2 M2 预处理与特征工程模块

该模块负责将原始信号转换为可训练输入。设计要点如下：

1. 使用滑动窗口将长序列切分为样本窗口。
2. 支持归一化、标准化等基础变换。
3. 支持时域和频域特征提取。
4. 预处理管道可按步骤组合，避免固定死流程。

1.5.3 3.3 M3 退化阶段划分与标签构造模块

该模块负责为不同任务生成目标值。设计要点如下：

1. RUL 标签构造面向回归任务。
2. 3sigma 和 FPT 面向阶段分类任务。
3. 健康指标序列构造面向滚动预测任务。

1.5.4 3.4 M4 模型训练与预测模块

该模块是系统核心。设计要点如下：

1. 模型统一继承基础接口，暴露输入输出维度和监控状态。

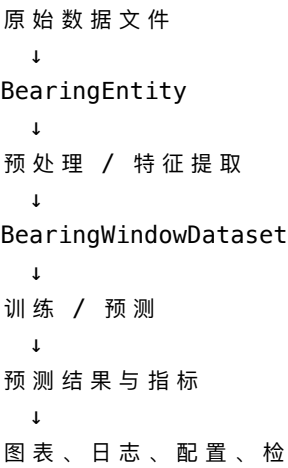
2. 训练器负责批量训练、验证和回调触发。
3. 预测器负责直接预测、滚动预测和不确定性估计。

1.5.5 3.5 M5 评估、可视化与实验管理模块

1. 评估器聚合多个指标。
2. 可视化模块输出图表。
3. 实验管理模块统一记录配置、历史、日志、模型和结果。

1.6 4. 数据流设计

系统数据流从原始数据到最终产物依次经过以下阶段：



设计上要求每一步都能被导出或检查，避免出现“中间状态不可见”的问题。

1.7 5. 数据库与数据文件设计

项目不引入关系型数据库，主要使用目录式文件组织。

类型	说明	典型格式
原始数据	官方数据集压缩包或解压目录	zip、rar、csv
中间数据	标签化数据集、缓存特征	csv、pkl
训练产物	模型检查点、历史记录、配置	pt、json、yaml、csv
结果图表	曲线图、热图、混淆矩阵	png

这种设计便于课程项目交付，也降低了数据库维护成本。

1.8 6. 核心算法设计

1.8.1 6.1 预处理算法

1. 滑动窗口：按照窗口长度和步长切分序列。
2. 归一化：在窗口级或样本级进行缩放。
3. 频域分析：对窗口做 FFT 并提取主频和能量。

1.8.2 6.2 退化阶段划分算法

1. 3sigma：基于健康指标偏离基线的统计阈值定义阶段。
2. FPT：识别首次稳定偏离健康状态的时刻，作为可预测点。

1.8.3 6.3 预测算法

1. CNN: 提取局部退化模式。
2. RNN: 建模时间依赖关系。
3. Transformer: 通过注意力建模中长序列关系。
4. Monte Carlo Dropout: 多次采样获得均值和方差。

1.9 7. 接口设计

1.9.1 7.1 数据接口

1. loader 接口: `list_entities()`、`load_entity(entity_id)`
2. dataset 接口: `split_by_ratio()`、`export()`

1.9.2 7.2 训练接口

1. trainer 接口: `train(model, train_set, valid_set)`
2. tester 接口: `test(model, test_set)`
3. callback 接口: 训练开始、批次结束、epoch 结束等生命周期钩子

1.9.3 7.3 评估与展示接口

1. evaluator: 统一注册多个指标并执行评估
2. visualizer: 导出图表文件和展示结果

1.10 8. 系统运行流程设计

一次标准运行流程如下:

1. 选择数据集并加载实体。
2. 选择预处理参数和标签构造方式。
3. 划分训练集和测试集。
4. 选择模型、训练器和回调。
5. 训练并记录过程。
6. 运行测试与评估。
7. 导出图表、日志和模型文件。

1.11 9. 异常处理设计

场景	处理方式
数据路径不存在	抛出明确异常并提示路径
文件列格式异常	尝试数值化并过滤非法行, 失败时给出说明
训练过程中梯度异常	记录告警并输出日志
导出目录不存在	自动创建目录后再导出

异常处理设计原则是“尽可能给出明确错误, 不做静默吞错”。

1.12 10. 部署设计

项目部署方式以本地运行和课程演示为主。

1. 通过 `uv` 安装依赖并创建环境。
2. 通过 `uv run python main.py` 运行主示例。
3. 通过 `uv run pytest -q` 运行自动化测试。
4. 通过 Pandoc 将文档导出为 PDF。

1.13 11. 设计总结

本设计文档的核心思想是以统一数据抽象和统一训练框架为中心，将数据、模型、评估和展示有机串联起来。这样既能满足课程项目的规范要求，也便于后续继续扩展新的算法和数据源。